0.1 First Fit Memory Allocation

First Fit searches memory sequentially from the first block (index 0) to the last block ($n-1$) and allocates the process into the **first block** whose capacity is greater than or equal to the requested size. Once allocated, the block size is decreased by the requested amount.

File: FA_first_fit.cpp

```cpp
#include <iostream>
#include <vector>

using namespace std;

#define ll long long
#define vi vector<int>
#define vll vector<long long>
#define pb push_back
#define all(x) (x).begin(), (x).end()

int main() {
    int n;
    cout << "Enter number of memory blocks: ";
    cin >> n;

    vi memory(n);
    for (int i = 0; i < n; i++) {
        cout << "Enter size of block " << (i + 1) << ": ";
        cin >> memory[i];
    }

    while (1) {
        int size, allocated = 0;
        cout << "\nEnter size needed (0 to exit): ";
        cin >> size;

        if (size == 0) break;

        for (int i = 0; i < n; i++) {
            if (memory[i] >= size) {
                memory[i] -= size;
                cout << "Allocated in block " << (i + 1) << "\n";
                allocated = 1;
                break;
            }
        }

        if (!allocated)
            cout << "No suitable block found\n";
    }

    return 0;
}
```

rahat@nixos:~/Downloads/Os -- ./FA_first_fit

```
rahat@nixos:~/Downloads/Os$ ./FA_first_fit
Enter number of memory blocks: 5
Enter size of block 1: 100
Enter size of block 2: 500
Enter size of block 3: 200
Enter size of block 4: 300
Enter size of block 5: 600

Enter size needed (0 to exit): 212
Allocated in block 2

Enter size needed (0 to exit): 417
Allocated in block 5

Enter size needed (0 to exit): 112
Allocated in block 2

Enter size needed (0 to exit): 426
No suitable block found

Enter size needed (0 to exit): 0
rahat@nixos:~/Downloads/Os$
```

0.2 Best Fit Memory Allocation

Best Fit examines all available memory blocks and allocates the process into the **smallest sufficient block** ($\geq$ size). This strategy minimizes leftover space inside the allocated block, reducing immediate internal fragmentation.

File: FA_best_fit.cpp

```cpp
#include <iostream>
#include <vector>

using namespace std;

#define ll long long
#define vi vector<int>
#define vll vector<long long>
#define pb push_back
#define all(x) (x).begin(), (x).end()

int main() {
    int n;
    cout << "Enter number of memory blocks: ";
    cin >> n;

    vi memory(n);
    for (int i = 0; i < n; i++) {
        cout << "Enter size of block " << (i + 1) << ": ";
        cin >> memory[i];
    }

    while (1) {
        int size, bestIndex = -1;
        cout << "\nEnter size needed (0 to exit): ";
        cin >> size;

        if (size == 0) break;

        for (int i = 0; i < n; i++) {
            if (memory[i] >= size) {
                if (bestIndex == -1 || memory[i] < memory[bestIndex]) {
                    bestIndex = i;
                }
            }
        }

        if (bestIndex != -1) {
            memory[bestIndex] -= size;
            cout << "Allocated in block " << (bestIndex + 1) << "\n";
        } else {
            cout << "No suitable block found\n";
        }
    }

    return 0;
}
```

rahat@nixos:~/Downloads/Os -- ./FA_best_fit

```
rahat@nixos:~/Downloads/Os$ ./FA_best_fit
Enter number of memory blocks: 5
Enter size of block 1: 100
Enter size of block 2: 500
Enter size of block 3: 200
Enter size of block 4: 300
Enter size of block 5: 600

Enter size needed (0 to exit): 212
Allocated in block 4

Enter size needed (0 to exit): 417
Allocated in block 2

Enter size needed (0 to exit): 112
Allocated in block 3

Enter size needed (0 to exit): 426
Allocated in block 5

Enter size needed (0 to exit): 0
rahat@nixos:~/Downloads/Os$
```

0.3 Worst Fit Memory Allocation

Worst Fit examines all memory blocks and allocates into the **largest available block**. The rationale is that the remaining leftover chunk will be large enough to remain useful for subsequent allocations.

File: FA_worst_fit.cpp

```cpp
#include <iostream>
#include <vector>

using namespace std;

#define ll long long
#define vi vector<int>
#define vll vector<long long>
#define pb push_back
#define all(x) (x).begin(), (x).end()

int main() {
    int n;
    cout << "Enter number of memory blocks: ";
    cin >> n;

    vi memory(n);
    for (int i = 0; i < n; i++) {
        cout << "Enter size of block " << (i + 1) << ": ";
        cin >> memory[i];
    }

    while (1) {
        int size, worstIndex = -1;
        cout << "\nEnter size needed (0 to exit): ";
        cin >> size;

        if (size == 0) break;

        for (int i = 0; i < n; i++) {
            if (memory[i] >= size) {
                if (worstIndex == -1 || memory[i] > memory[worstIndex]) {
                    worstIndex = i;
                }
            }
        }

        if (worstIndex != -1) {
            memory[worstIndex] -= size;
            cout << "Allocated in block " << (worstIndex + 1) << "\n";
        } else {
            cout << "No suitable block found\n";
        }
    }

    return 0;
}
```

rahat@nixos:~/Downloads/Os -- ./FA_worst_fit

```
rahat@nixos:~/Downloads/Os$ ./FA_worst_fit
Enter number of memory blocks: 5
Enter size of block 1: 100
Enter size of block 2: 500
Enter size of block 3: 200
Enter size of block 4: 300
Enter size of block 5: 600

Enter size needed (0 to exit): 212
Allocated in block 5

Enter size needed (0 to exit): 417
Allocated in block 2

Enter size needed (0 to exit): 112
Allocated in block 5

Enter size needed (0 to exit): 426
No suitable block found

Enter size needed (0 to exit): 0
rahat@nixos:~/Downloads/Os$
```

0.4 Next Fit Memory Allocation

Next Fit is a modification of First Fit that starts scanning from the **last allocated position** instead of restarting from the beginning. It cycles through the memory array using modular index progression: `i = (lastIndex + count) % n`.

File: FA_next_fit.cpp

```cpp
#include <iostream>
#include <vector>

using namespace std;

#define ll long long
#define vi vector<int>
#define vll vector<long long>
#define pb push_back
#define all(x) (x).begin(), (x).end()

int main() {
    int n;
    cout << "Enter number of memory blocks: ";
    cin >> n;

    vi memory(n);
    for (int i = 0; i < n; i++) {
        cout << "Enter size of block " << (i + 1) << ": ";
        cin >> memory[i];
    }

    int lastIndex = 0;

    while (1) {
        int size, allocated = 0;
        cout << "\nEnter size needed (0 to exit): ";
        cin >> size;

        if (size == 0) break;

        for (int count = 0; count < n; count++) {
            int i = (lastIndex + count) % n;
            if (memory[i] >= size) {
                memory[i] -= size;
                cout << "Allocated in block " << (i + 1) << "\n";
                lastIndex = i;
                allocated = 1;
                break;
            }
        }

        if (!allocated)
            cout << "No suitable block found\n";
    }

    return 0;
}
```

rahat@nixos:~/Downloads/Os -- ./FA_next_fit

```
rahat@nixos:~/Downloads/Os$ ./FA_next_fit
Enter number of memory blocks: 5
Enter size of block 1: 100
Enter size of block 2: 500
Enter size of block 3: 200
Enter size of block 4: 300
Enter size of block 5: 600

Enter size needed (0 to exit): 212
Allocated in block 2

Enter size needed (0 to exit): 417
Allocated in block 5

Enter size needed (0 to exit): 112
Allocated in block 5

Enter size needed (0 to exit): 426
No suitable block found

Enter size needed (0 to exit): 0
rahat@nixos:~/Downloads/Os$
```

1 Process Scheduling Algorithms

1.1 First-Come, First-Served (FCFS / FIFO) Scheduling

FIFO schedules processes strictly in the order of their arrival times. Processes are sorted by arrival time, and execution proceeds non-preemptively. Turnaround time (TAT = CT − AT) and waiting time (WT = TAT − BT) are calculated and averaged.

File: PS_FIFO.cpp

```cpp
#include <iomanip>
#include <iostream>
#include <vector>

using namespace std;

#define ll long long
#define vi vector<int>
#define vll vector<long long>
#define pb push_back
#define all(x) (x).begin(), (x).end()

int main() {
    int n;
    cout << "Enter the number of processes: ";
    cin >> n;

    vi process(n), arrival(n), execution(n), completion(n);
    vi turnaround(n), waiting(n);

    int i, j, pos, temp;
    float averageTAT = 0, averageWT = 0;

    for (i = 0; i < n; i++) {
        process[i] = i + 1;

        cout << "Enter arrival time of process " << process[i] << ": ";
        cin >> arrival[i];

        cout << "Enter execution time of process " << process[i] << ": ";
        cin >> execution[i];

        completion[i] = -1;
    }

    for (i = 0; i < n; i++) {
        pos = i;

        for (j = i + 1; j < n; j++) {
            if (arrival[pos] > arrival[j]) {
                pos = j;
            }
        }

        temp = arrival[i];
        arrival[i] = arrival[pos];
        arrival[pos] = temp;

        temp = execution[i];
        execution[i] = execution[pos];
        execution[pos] = temp;

        temp = process[i];
        process[i] = process[pos];
        process[pos] = temp;
    }

    for (i = 0; i < n; i++) {
        if (i == 0) {
            completion[i] = arrival[i] + execution[i];
        }
        else {
            if (completion[i - 1] > arrival[i]) {
```

```
            completion[i] = completion[i - 1] + execution[i];
        }
        else {
            completion[i] = arrival[i] + execution[i];
        }
    }
}

for (i = 0; i < n; i++) {
    turnaround[i] = completion[i] - arrival[i];
    waiting[i] = turnaround[i] - execution[i];

    averageTAT = averageTAT + turnaround[i];
    averageWT = averageWT + waiting[i];
}

averageTAT = averageTAT / n;
averageWT = averageWT / n;

cout << "\nProcess\tArrival\tExecution\tCompletion\tTAT\tWT\n";

for (i = 0; i < n; i++) {
    cout << "P" << process[i] << "\t"
         << arrival[i] << "\t"
         << execution[i] << "\t\t"
         << completion[i] << "\t\t"
         << turnaround[i] << "\t"
         << waiting[i] << "\n";
}

cout << "\nAverage Turnaround Time = " << fixed << setprecision(2) << averageTAT;
cout << "\nAverage Waiting Time = " << fixed << setprecision(2) << averageWT << "\n";

return 0;
}
```

rahat@nixos:~/Downloads/Os -- ./PS_FIFO

```
rahat@nixos:~/Downloads/Os$ ./PS_FIFO
Enter the number of processes: 4
Enter arrival time of process 1: 0
Enter execution time of process 1: 5
Enter arrival time of process 2: 1
Enter execution time of process 2: 3
Enter arrival time of process 3: 2
Enter execution time of process 3: 8
Enter arrival time of process 4: 3
Enter execution time of process 4: 6

Process Arrival Execution       Completion      TAT     WT
P1      0       5               5               5       0
P2      1       3               8               7       4
P3      2       8               16              14      6
P4      3       6               22              19      13

Average Turnaround Time = 11.25
Average Waiting Time    = 5.75
rahat@nixos:~/Downloads/Os$
```

1.2 Shortest Job First (SJF) Scheduling (Non-Preemptive)

Shortest Job First schedules the arrived process with the smallest CPU burst time. If burst times tie, the process with the earlier arrival time is scheduled first. Once scheduled, the CPU runs the process to completion.

File: PS_Shortest_Job_first.cpp

```cpp
#include <iomanip>
#include <iostream>
#include <vector>

using namespace std;

#define ll long long
#define vi vector<int>
#define vll vector<long long>
#define pb push_back
#define all(x) (x).begin(), (x).end()

int main() {
    int n;

    cout << "Enter the number of processes: ";
    cin >> n;

    vi process(n), arrival(n), burst(n);
    vi completion(n), turnaround(n), waiting(n);
    vi is_completed(n);

    int i, j, temp, pos;
    int current_time = 0;
    int completed = 0;

    float totalTAT = 0;
    float totalWT = 0;
    float averageTAT, averageWT;

    for (i = 0; i < n; i++) {
        process[i] = i + 1;

        cout << "Enter arrival time of process " << process[i] << ": ";
        cin >> arrival[i];

        cout << "Enter burst time of process " << process[i] << ": ";
        cin >> burst[i];

        completion[i] = -1;
        is_completed[i] = 0;
    }

    for (i = 0; i < n; i++) {
        pos = i;

        for (j = i + 1; j < n; j++) {
            if (arrival[j] < arrival[pos]) {
                pos = j;
            }
        }

        temp = arrival[i];
        arrival[i] = arrival[pos];
        arrival[pos] = temp;

        temp = burst[i];
        burst[i] = burst[pos];
        burst[pos] = temp;

        temp = process[i];
        process[i] = process[pos];
        process[pos] = temp;
    }

    while (completed != n) {
```

```
        int min_burst = 9999;
        int index = -1;

        for (i = 0; i < n; i++) {
            if (arrival[i] <= current_time && is_completed[i] == 0) {
                if (burst[i] < min_burst) {
                    min_burst = burst[i];
                    index = i;
                }
                else if (burst[i] == min_burst &&
                        arrival[i] < arrival[index]) {
                    index = i;
                }
            }
        }

        if (index != -1) {
            completion[index] = current_time + burst[index];
            turnaround[index] = completion[index] - arrival[index];
            waiting[index] = turnaround[index] - burst[index];

            totalTAT = totalTAT + turnaround[index];
            totalWT = totalWT + waiting[index];

            current_time = completion[index];

            is_completed[index] = 1;
            completed++;
        }
        else {
            current_time++;
        }
    }

    averageTAT = totalTAT / n;
    averageWT = totalWT / n;

    cout << "\nProcess\tArrival Time\tBurst Time\tCompletion Time\tTurnaround Time\tWaiting Time\n";

    for (i = 0; i < n; i++) {
        cout << "P" << process[i] << "\t"
             << arrival[i] << "\t\t"
             << burst[i] << "\t\t"
             << completion[i] << "\t\t"
             << turnaround[i] << "\t\t"
             << waiting[i] << "\n";
    }

    cout << "\nTotal Turnaround Time = " << fixed << setprecision(0) << totalTAT;
    cout << "\nTotal Waiting Time = " << fixed << setprecision(0) << totalWT;

    cout << "\nAverage Turnaround Time = " << fixed << setprecision(2) << averageTAT;
    cout << "\nAverage Waiting Time = " << fixed << setprecision(2) << averageWT << "\n";

    return 0;
}
```

rahat@nixos:~/Downloads/Os -- ./PS_Shortest_Job_first

```
rahat@nixos:~/Downloads/Os$ ./PS_Shortest_Job_first
Enter the number of processes: 4
Enter arrival time of process 1: 0
Enter burst time of process 1: 7
Enter arrival time of process 2: 2
Enter burst time of process 2: 4
Enter arrival time of process 3: 4
Enter burst time of process 3: 1
Enter arrival time of process 4: 5
Enter burst time of process 4: 4

Process Arrival Time    Burst Time      Completion Time Turnaround Time Waiting Time
P1      0               7               7               7               0
P2      2               4               12              10              6
P3      4               1               8               4               3
P4      5               4               16              11              7

Total Turnaround Time = 32
Total Waiting Time    = 16
Average Turnaround Time = 8.00
Average Waiting Time    = 4.00
rahat@nixos:~/Downloads/Os$
```

1.3 Round Robin (RR) Scheduling

Round Robin allocates CPU time slices of a fixed Time Quantum (TQ) in circular order. Arrived processes enter a FIFO queue. If a process does not finish within TQ, it is preempted, returned to the end of the queue, and scheduled again in subsequent rounds.

File: PS_Roundrobin.cpp

```cpp
#include <iomanip>
#include <iostream>
#include <vector>

using namespace std;

#define ll long long
#define vi vector<int>
#define vll vector<long long>
#define pb push_back
#define all(x) (x).begin(), (x).end()

int main() {
    int n, i, tq;

    cout << "Enter the number of processes: ";
    cin >> n;

    vi process(n), arrival(n), burst(n), remaining(n), completion(n);
    vi waiting(n), turnaround(n);

    for (i = 0; i < n; i++) {
        process[i] = i + 1;
        cout << "Enter arrival time of process " << process[i] << ": ";
        cin >> arrival[i];
        cout << "Enter burst time of process " << process[i] << ": ";
        cin >> burst[i];

        remaining[i] = burst[i];
        completion[i] = 0;
    }

    cout << "Enter Time Quantum: ";
    cin >> tq;

    int time = 0, completed = 0;
    int queue[100], front = 0, rear = 0;

    vi visited(n);
    for (i = 0; i < n; i++)
        visited[i] = 0;

    for (i = 0; i < n; i++) {
        if (arrival[i] == 0) {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }

    while (completed < n) {
        if (front == rear) {
            int next_arrival = 9999;

            for (i = 0; i < n; i++) {
                if (remaining[i] > 0 && arrival[i] > time && arrival[i] < next_arrival) {
                    next_arrival = arrival[i];
                }
            }

            time = next_arrival;

            for (i = 0; i < n; i++) {
                if (arrival[i] <= time && remaining[i] > 0 && !visited[i]) {
                    queue[rear++] = i;
                    visited[i] = 1;
                }
```

```
            }
        }

        int idx = queue[front++];

        if (remaining[idx] > tq) {
            time += tq;
            remaining[idx] -= tq;
        } else {
            time += remaining[idx];
            remaining[idx] = 0;
            completion[idx] = time;
            completed++;
        }

        for (i = 0; i < n; i++) {
            if (arrival[i] <= time && remaining[i] > 0 && !visited[i]) {
                queue[rear++] = i;
                visited[i] = 1;
            }
        }

        if (remaining[idx] > 0) {
            queue[rear++] = idx;
        }
    }

    float total_wt = 0, total_tat = 0;

    cout << "\nProcess\tArrival\tBurst\tCompletion\tTurnaround\tWaiting\n";
    for (i = 0; i < n; i++) {
        turnaround[i] = completion[i] - arrival[i];
        waiting[i] = turnaround[i] - burst[i];

        total_wt += waiting[i];
        total_tat += turnaround[i];

        cout << process[i] << "\t"
             << arrival[i] << "\t"
             << burst[i] << "\t"
             << completion[i] << "\t\t"
             << turnaround[i] << "\t\t"
             << waiting[i] << "\n";
    }

    cout << "\nAverage Waiting Time: " << fixed << setprecision(2) << (total_wt / n);
    cout << "\nAverage Turnaround Time: " << fixed << setprecision(2) << (total_tat / n) << "\n";

    return 0;
}
```

rahat@nixos:~/Downloads/Os -- ./PS_Roundrobin

```
rahat@nixos:~/Downloads/Os$ ./PS_Roundrobin
Enter the number of processes: 4
Enter arrival time of process 1: 0
Enter burst time of process 1: 5
Enter arrival time of process 2: 1
Enter burst time of process 2: 4
Enter arrival time of process 3: 2
Enter burst time of process 3: 2
Enter arrival time of process 4: 3
Enter burst time of process 4: 1
Enter Time Quantum: 2

Process Arrival Burst   Completion      Turnaround      Waiting
1       0       5       12              12              7
2       1       4       11              10              6
3       2       2       6               4               2
4       3       1       9               6               5

Average Waiting Time: 5.00
Average Turnaround Time: 8.00
rahat@nixos:~/Downloads/Os$
```

1.4 Shortest Remaining Time First (SRTF) Scheduling

Shortest Remaining Time First is the preemptive counterpart of SJF. At each scheduling step, the process with the lowest remaining execution time among arrived processes is selected to execute for up to the time quantum before re-evaluating the shortest remaining job.

File: PS_Shortest_remaining.cpp

```cpp
#include <iomanip>
#include <iostream>
#include <vector>

using namespace std;

#define ll long long
#define vi vector<int>
#define vll vector<long long>
#define pb push_back
#define all(x) (x).begin(), (x).end()

int main() {
    int n, i, tq;
    cout << "Enter the number of processes: ";
    cin >> n;

    vi process(n), arrival(n), burst(n), remaining(n), completion(n);
    vi waiting(n), turnaround(n);

    for (i = 0; i < n; i++) {
        process[i] = i + 1;
        cout << "Enter arrival time of process " << process[i] << ": ";
        cin >> arrival[i];
        cout << "Enter burst time of process " << process[i] << ": ";
        cin >> burst[i];

        remaining[i] = burst[i];
        completion[i] = 0;
    }

    cout << "Enter Time Quantum: ";
    cin >> tq;

    int time = 0, completed = 0;

    while (completed < n) {
        int min_index = -1;
        int min_remaining = 9999;

        for (i = 0; i < n; i++) {
            if (arrival[i] <= time && remaining[i] > 0 && remaining[i] < min_remaining) {
                min_remaining = remaining[i];
                min_index = i;
            }
        }

        if (min_index == -1) {
            int next_arrival = 9999;
            for (i = 0; i < n; i++) {
                if (arrival[i] > time && arrival[i] < next_arrival)
                    next_arrival = arrival[i];
            }
            time = next_arrival;
            continue;
        }

        int exec_time = (remaining[min_index] > tq) ? tq : remaining[min_index];
        remaining[min_index] -= exec_time;
        time += exec_time;

        if (remaining[min_index] == 0) {
            completion[min_index] = time;
            completed++;
        }
    }
```

```
67
68      float total_wt = 0, total_tat = 0;
69
70      cout << "\nProcess\tArrival\tBurst\tCompletion\tTurnaround\tWaiting\n";
71      for (i = 0; i < n; i++) {
72          turnaround[i] = completion[i] - arrival[i];
73          waiting[i] = turnaround[i] - burst[i];
74
75          total_wt += waiting[i];
76          total_tat += turnaround[i];
77
78          cout << process[i] << "\t"
79               << arrival[i] << "\t"
80               << burst[i] << "\t"
81               << completion[i] << "\t\t"
82               << turnaround[i] << "\t\t"
83               << waiting[i] << "\n";
84      }
85
86      cout << "\nAverage Waiting Time: " << fixed << setprecision(2) << (total_wt / n);
87      cout << "\nAverage Turnaround Time: " << fixed << setprecision(2) << (total_tat / n) << "\n";
88
89      return 0;
90  }
```

rahat@nixos:~/Downloads/Os -- ./PS_Shortest_remaining

```
rahat@nixos:~/Downloads/Os$ ./PS_Shortest_remaining
Enter the number of processes: 4
Enter arrival time of process 1: 0
Enter burst time of process 1: 5
Enter arrival time of process 2: 1
Enter burst time of process 2: 4
Enter arrival time of process 3: 2
Enter burst time of process 3: 2
Enter arrival time of process 4: 3
Enter burst time of process 4: 1
Enter Time Quantum: 2

Process Arrival Burst   Completion      Turnaround      Waiting
1       0       5       8               8               3
2       1       4       12              11              7
3       2       2       4               2               0
4       3       1       5               2               1

Average Waiting Time: 2.75
Average Turnaround Time: 5.75
rahat@nixos:~/Downloads/Os$
```